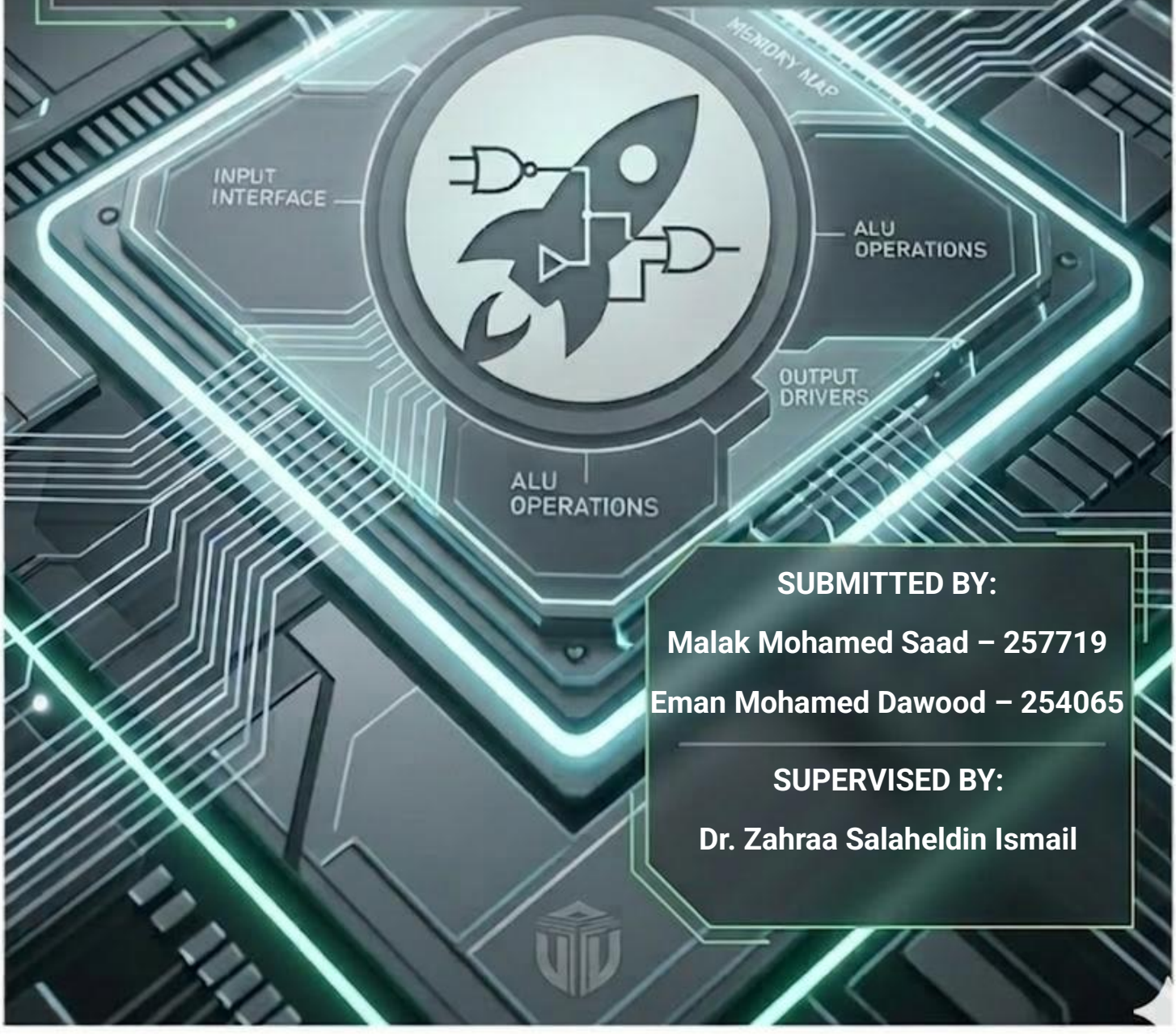
ROCKET LAUNCH CONTROL PANEL

The BRITISH UNIVERSITY IN EGYPT

Module Name: Computer Organization

Module Code: 25CPE4402



SUBMITTED BY:

Malak Mohamed Saad – 257719

Eman Mohamed Dawood – 254065

SUPERVISED BY:

Dr. Zahraa Salaheldin Ismail

Table of Contents

List of Figures

List of Tables

1. System Overview

The **Rocket Launch Control Panel** is a simulated control system implemented using hardware-based design concepts targeting an **FPGA** platform. The system introduces a simple model of a real rocket launch control panel where multiple subsystems must be activated and verified before the rocket ignition and launch process can start.

The system ensures that all required subsystems are properly enabled and sufficient before ignition and launch. These subsystems include the **fuel system**, **engine system**, and **safety lock**. In addition, the system validates a **launch authorization code** to confirm that the launch command is approved.

The operator can initiate the rocket launch sequence, once all required subsystems conditions are satisfied. The system then starts performing multiple sequential operations, which are:

- Countdown sequence from **10 to 1**
- Rocket ignition process starts
- Telemetry monitoring after launch

The system also supports an **emergency abort function** that allows the operator to immediately stop the launch at any time if a critical issue occurs. When the abort command is triggered, the launch sequence is deactivated, an alarm sound is activated, and the mission is terminated safely.

During launching, the system simulates the telemetry data of the rocket like **speed**, **altitude**, and **fuel level**, which update continuously and displayed to represent the flight status of the rocket.

This project demonstrates how a **microprocessor-based control system** can manage and coordinate multiple subsystems involved in a complex real-world process like launching a rocket. The system was implemented using **Xilinx Artix-7 FPGA (Basys 3 board)** and simulated using **Irvine32 library** to illustrate the interaction between safety control, control logic, operational sequencing, and system verification.

2. System Features

The implemented system supports the following features:

1. System Control Panel

A keyboard-controlled interface allows activating multiple subsystems.

Available commands:

Key	Function
F	Enable Fuel System
E	Enable Engine System
S	Activate Safety Lock
L	Launch Rocket
A	Abort Mission

Table 1: Function of each key

2. Safety Verification

Before launching, the system ensures that the:

- Fuel system is enabled
- Engine system is ready
- Safety lock – A safety mechanism which prevents accidental launch unless all required conditions are satisfied.

If any condition is missing:

Launch Blocked – System Not Ready

3. Authorization System

Before launch, the operator must enter an **authorization code** (e.g., 1234).

If incorrect:

- Access denied
- Alarm sound activated
- The system prompts the user to enter the authorization code again, allowing another attempt until the correct code is provided.

If correct:

- Access is granted
- Countdown begins

4. Countdown System

The system starts a **10-second countdown** before ignition.

During countdown:

- Numbers are displayed
- Voice announcement is generated

5. Abort System

The system supports an **emergency abort** at any time.

When triggered:

- Launch is cancelled
- Alarm sound plays
- Program terminates

6. Rocket Ignition System

When countdown finishes an Ignition message and Rocket launch success message appear

7. Telemetry Monitoring

The system simulates rocket telemetry:

Parameter	Description
Altitude	Increases during launch
Speed	Increases
Fuel	Decreases

Table 2: Telemetry parameters description

3. Simulation Environment

The rocket launch control system was simulated using the **Irvine32 Library**, which provides I/O procedures for assembly programs.

The simulation was executed on a PC environment where:

- keyboard input represents control panel buttons
- console output represents system displays
- sound functions simulate alarm signals

This environment enables testing of the system logic without requiring actual hardware.

---

4. Program Design / Software Architecture

This section presents the internal and low-level implementation of the system using ×86 assembly language. It highlights how core computer organization concepts such as registers, memory management, use of variables, control flow instructions, and modular programming are applied to achieve the required functionality and construct a reliable and structured control system.

A state variables where used like **fuelState**, **engineState**, and **safetyState** to show the status of each subsystem (Fuel / Engine / Safety). These variables are stored in memory and updated dynamically based on the user input using the keyboard, acting as a signal to determine if the system ready or not.

```
fuelState   BYTE 0
engineState BYTE 0
safetyState BYTE 0
```

The input of the user is handled by **ReadKey** procedure, which stores the key pressed temporarily in the AL register and then compared against predefined commands using conditional branching to jump to the instruction that matches the key to perform its corresponding action.

```
MainLoop:
    call ReadKey
    jz   MainLoop

    cmp al, 'f'
    je  FuelOn
    cmp al, 'e'
    je  EngineOn
    cmp al,'s'
    je SafetyOn
    cmp al, 'l'
    je  Launch
    cmp al, 'a'
    je  AbortNow
    jmp MainLoop
```

Decision-making is implemented using compare instruction **CMP**, which then decides what to do next using conditional jumps such as **je** (jump if equal) and **jne** (jump if not equal) [1]. For example, before prompting the auth code and starting the countdown, the system ensures that all subsystem states are active. If any condition fails, execution jumps to another function which is an error-handling part that blocks the launch.

```
cmp fuelState, 1
jne Block
cmp engineState, 1
jne Block
cmp safetyState,1
jne Block
```

Iterative operations such as the countdown sequence and updating telemetry data are implemented using loops based on conditional jumps. These loops continue running until specific termination conditions are met or a certain tells the program to stop.

```
CountdownLoop:
     call CheckAbort
     ; Print current number
     mov eax, countVal
     call WriteDec
     call Crlf

     cmp eax,10
     je SpeakTen
     cmp eax,9
     je SpeakNine
     cmp eax,8
     je SpeakEight
     cmp eax,7
     je SpeakSeven
     cmp eax,6
     je SpeakSix
     cmp eax,5
     je SpeakFive
     cmp eax,4
     je SpeakFour
     cmp eax,3
     je SpeakThree
     cmp eax,2
     je SpeakTwo
     cmp eax,1
     je SpeakOne

AfterSpeak:

     mov eax,1000
     call Delay
     dec countVal
     cmp countVal,0
     jg CountdownLoop
     call Ignition
     jmp MainLoop
```

```
; ---- Fuel ----
mov dh,31
mov dl,0
call Gotoxy
mov edx, OFFSET fuelLabel
call WriteString
mov eax, fuel
call WriteDec
mov edx, OFFSET percent
call WriteString
mov edx, OFFSET spaces
call WriteString

mov eax,1000
call Delay

cmp fuel,10
jg TelemetryLoop
```

The program is split into modular procedures (separate blocks of code) such as **ShowTelemetry** which displays telemetry data (fuel + speed + altitude), **CheckAbort** which checks if the abort key “a” is pressed, and **Ignition** which starts the engine to start launching, each responsible for a specific functionality instead if one huge messy list of instructions. This modular design improves the code maintainability and readability.

```
ShowTelemetry PROC
CheckAbort PROC
Ignition PROC
Alarm PROC
```

Registers such as **eax** and **edx** are used for storing data and numbers temporarily while the program executes and help in passing parameters between procedures [1].

The system follows a **finite state machine (FSM)** model, where the program has a set of allowed modes and strict rules for transition [1] between states such as idle, verification, authorization, countdown, and launch. This structured approach ensures controlled execution flow and prevents invalid system behavior and illegal actions.

5. I/O Devices Selection and I/O Interface Design

In a real-world implementation, the Rocket Launch Control Panel System would interact with several hardware devices connected to the **Xilinx Artix-7 FPGA**. These devices are responsible for receiving input from sensors and operators and providing output signals to control subsystems and display system status.

The devices used in the system can be classified into **input devices** and **output devices**.

5.1 Input Devices

Input devices provide information to the microprocessor about the current system status or allow the operator to control the launch process.

Input Device	Model Used	Function
Control Panel Buttons	Basys 3 Built-in Buttons	Used to control system operations such as **Launch** and **Abort** commands
Keypad	4×4 Matrix Keypad	Allows the operator to enter the **launch authorization code**
Fuel Level Sensor	HC-SR04 Ultrasonic Sensor	Measures the amount of fuel available in the rocket tank
Altitude Sensor	BMP280	Measures the rocket altitude during flight
Speed Sensor	Hall Effect Sensor (A3144)	Measures the rocket velocity
Temperature Sensor Pressure Sensor	BME280	Monitors **engine temperature** Monitors **fuel line pressure**, and **mechanical stability**

Table 3: Input devices and its function

5.2 Output Devices

Output devices display system information and activate the different subsystems required for the rocket launch process.

Output Device	Model Used	Function
Fuel System Control	SRD-05VDC-SL-C Relay	Enables or disables the rocket fuel system
Engine System Control	SRD-05VDC-SL-C Relay	Activates the engine system

Safety Lock	Solenoid Lock	Activates the safety interlock mechanism
LCD Display	I2C LCD	Displays system messages and telemetry data
LED Indicators	5mm LEDs	Indicate system status such as system ready or errors
Buzzer	Active Piezo	Produces alarm signals during errors or mission abort
Speaker	LM386 + 8Ω 0.5W Speaker	Generates voice notifications and alerts
7-Segment Display	Built-in Basys 3	Displays the launch countdown sequence

Table 4: Output devices and its function

5.3 I/O Connection to the Xilinx Artix-7 FPGA

In the proposed hardware implementation, the system is designed using an FPGA platform, which provides a large number of configurable I/O pins. These pins are programmed to interface directly with various system components, including sensors, control inputs, and output devices.

The FPGA implements custom digital logic to manage input processing and output control in real time. Input devices such as the keypad and sensors (e.g., fuel rate, temperature, and pressure sensors) provide digital signals to the FPGA through GPIO and communication interfaces like I2C. These inputs are processed by the internal logic to determine the system state and control decisions [3].

Output devices, including LEDs, displays, buzzers, and relay modules, are driven by the FPGA through its output pins [2]. Since some components require higher voltage or current levels, interface circuits such as relay drivers are used to safely control external subsystems like the fuel system, engine system, and safety lock.

This architecture ensures efficient data flow, real-time responsiveness, and reliable control, making it suitable for safety-critical embedded applications.

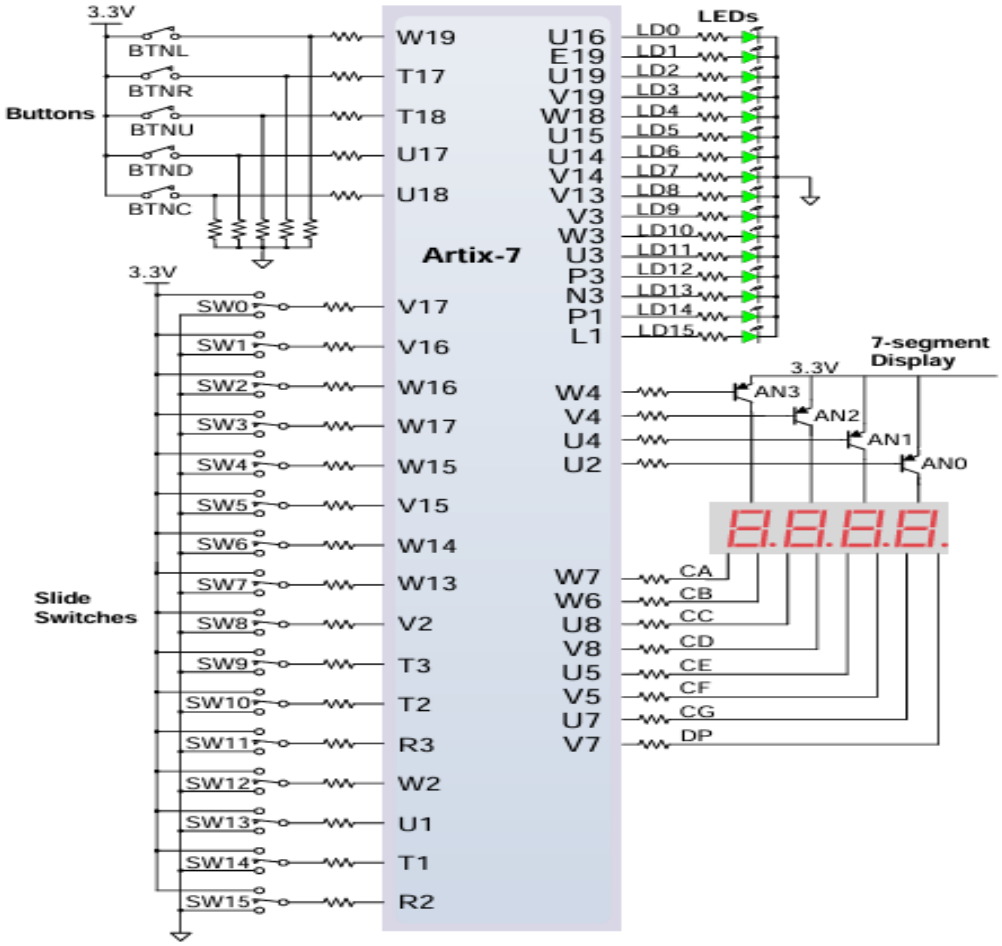


Figure 1: General purpose I/O devices on the Basys 3

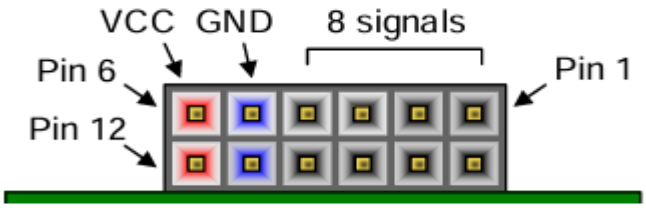


Figure 2: Pmod ports

Pmod JA	Pmod JB	Pmod JC	Pmod XDAC
JA1: J1	JB1: A14	JC1: K17	JXADC1: J3
JA2: L2	JB2: A16	JC2: M18	JXADC2: L3
JA3: J2	JB3: B15	JC3: N17	JXADC3: M2
JA4: G2	JB4: B16	JC4: P18	JXADC4: N2
JA7: H1	JB7: A15	JC7: L17	JXADC7: K3
JA8: K2	JB8: A17	JC8: M19	JXADC8: M3
JA9: H2	JB9: C15	JC9: P17	JXADC9: M1
JA10: G3	JB10: C16	JC10: R18	JXADC10: N1

Table 5: Basys 3 Pmod pin assignment

6. Alternative MCUs (Microcontroller Unit)

To evaluate possible alternatives to the FPGA-based implementation, two microcontroller units (MCUs) were analyzed: the **Raspberry Pi Pico (RP2040 Dual-core ARM Cortex-M0+)** and the **STM32F407VG**.
These devices were selected due to their strong processing capabilities, sufficient I/O interfaces, and widespread use in embedded control applications.

6.1 STM32F407VG

The **STM32F407VG** is a high-performance MCU designed for industrial and advanced embedded applications. It provides significantly greater processing power and advanced peripheral support compared to entry-level MCUs.

6.1.1 Key Specifications [8]

- **Processor:** ARM Cortex-M4 with Floating Point Unit (FPU)
- **Clock Speed:** Up to **168 MHz**
- **Performance:** Up to **210 DMIPS**
- **Memory:**
    - Up to **1 MB Flash Memory**
    - **192 KB SRAM**
- **GPIO Pins:** Up to **140 I/O pins**
- **Communication Interfaces:**
    - Up to 6 USART/UART
    - 3 SPI interfaces
    - 3 I2C interfaces
    - 2 CAN interfaces
    - USB 2.0 OTG
    - Ethernet MAC support
- **Analog Features:**

    - 3 × 12-bit ADCs
    - 2 × 12-bit DACs
- **Timers:** Up to 17 timers
- **Debug Interfaces:** SWD and JTAG
- **Additional Features:**
    - DSP instructions
    - Camera interface support
    - Hardware cryptographic acceleration
    - Real-time clock (RTC)

6.1.2 Advantages for This Project

The STM32F407VG offers several benefits for implementing a rocket launch control panel:

- The **Floating Point Unit (FPU)** allows fast mathematical computations, useful for telemetry processing.
- The large number of **GPIO pins** allows simultaneous connection of multiple sensors, displays, and actuators.
- Built-in **ADC and DAC modules** enable advanced signal processing and analog control.
- Its industrial-grade design makes it suitable for **real-time control systems**.

6.1.3 Limitations

Some limitations of the STM32F407VG include:

- Higher **cost** compared to simpler microcontrollers.
- Requires more **complex hardware setup**, including external clock circuits and voltage regulation.
- More advanced development environment compared to beginner-friendly platforms.

6.2 Raspberry Pi Pico (RP2040 Dual-core ARM Cortex-M0+)

The **Raspberry Pi Pico** is a low-cost and high-performance MCU development board based on the RP2040 MCU. It is designed to provide flexible digital interfacing and reliable performance for embedded control systems.

6.2.1 Key Specifications [6]

- **Processor:** Dual-core ARM Cortex-M0+
- **Clock Speed:** Up to 133 MHz
- **Memory:**
    - 264 KB on-chip SRAM
    - 2 MB external QSPI Flash
- **GPIO Pins:** 26 multi-function GPIO pins
- **Communication Interfaces:**
    - 2 × UART
    - 2 × SPI
    - 2 × I2C
- **PWM Channels:** 16 PWM outputs
- **Analog Inputs:** 3 × 12-bit ADC channels
- **USB Support:** USB 1.1 (Host/Device)
- **Additional Features:**
    - 8 Programmable I/O (PIO) state machines
    - On-chip temperature sensor
    - Low-power sleep and dormant modes
    - Drag-and-drop USB programming
- **Operating Temperature:** −40°C to +85°C

6.2.2 Advantages for This Project

The Raspberry Pi Pico is suitable for the rocket launch control panel because:

- It provides sufficient **GPIO pins** to connect input devices like sensors and keypads.
- It supports **PWM outputs**, which is useful for controlling alarms and buzzers [7].
- The **Programmable I/O (PIO)** allows implementing a custom digital interfaces if specialized hardware timing is required [7].
- The dual-core processor allows handling multiple system tasks efficiently.

6.2.3 Limitations

Despite its advantages, the Raspberry Pi Pico has some limitations:

- It does not include a **hardware floating-point unit (FPU)**, which reduces efficiency in complex calculations.
- It provides limited analog output capability, as it lacks **Digital-to-Analog Converters (DACs)**.

6.3 Final Evaluation of Alternative MCUs

Both the **Raspberry Pi Pico** and the **STM32F407VG** are capable of implementing the Rocket Launch Control Panel. Each device provides sufficient processing power and I/O capabilities to handle subsystem activation, authorization, countdown sequencing, and output control.

However, when comparing the two MCUs, the **STM32F407VG** is considered the more suitable and powerful option for this type of control system.

Although the **Raspberry Pi Pico** provides good performance at low cost, the **STM32F407VG** offers significantly higher capability, more communication interfaces, and advanced analog support, making it more appropriate for complex control systems.

6.3.1 Higher Processing Performance

The **STM32F407VG** operates at a higher maximum clock speed **(168 MHz)** compared to the **Raspberry Pi Pico (133 MHz)**. Additionally, the **STM32F407VG** includes a hardware **Floating Point Unit (FPU)** and **DSP** instructions, which

allows faster execution of mathematical calculations required for telemetry processing and system timing operations.

6.3.2 Greater Number of GPIO Pins

The **STM32F407VG** supports up to **140 GPIO pins**, while the **Raspberry Pi Pico** provides **26 GPIO pins**.
This larger number of I/O pins allows easier connection of multiple sensors, relays, displays, and control units without requiring additional expansion hardware.

6.3.3 Advanced Peripheral Support

The **STM32F407VG** includes advanced interfaces like:

- CAN communication
- Ethernet connectivity
- Multiple ADC and DAC modules
- High-performance timers

These features make it more suitable for industrial-grade control applications.

In conclusion, based on performance, flexibility, and peripheral capability, the **STM32F407VG** is considered the better MCU option for implementing the Rocket Launch Control Panel system compared to the **Raspberry Pi Pico**.

However, it is important to note that both MCUs are technically capable of implementing the required system functionality.

Although both MCUs are capable of implementing the system, the **Xilinx Artix-7 FPGA** was selected as the primary hardware platform due to its superior flexibility and performance in real-time control applications.

6.4 Advantages of FPGA Over Microcontrollers

6.4.1 Parallel Processing Capability

MCUs execute instructions sequentially, which means that the operations occur one after another.

In contrast, an FPGA allows true parallel execution, which means multiple operations like:

- Input monitoring
- Subsystem verification
- Abort detection
- Output control

can occur simultaneously (at the same time).

This significantly improves system responsiveness and reliability, which is critical for safety-sensitive systems.

6.4.2 Real-Time System Control

FPGA logic operates at the hardware level, which allows faster reaction to input signals compared to software-driven MCUs.

This ensures:

- Faster response to emergency conditions
- Immediate detection of abort signals
- Accurate timing during countdown operations

6.4.3 Greater Hardware Flexibility

Unlike MCUs, FPGA hardware can be fully customized to implement dedicated logic circuits.

This allows:

- Creation of custom control modules
- Implementation of optimized timing logic
- Flexible modification of system architecture

Such flexibility makes FPGA platforms highly suitable for complex embedded control systems.

6.4.4 High Reliability for Safety-Critical Systems

FPGA systems are widely used in aerospace and industrial applications as they provide:

- Deterministic timing behavior

- High reliability
- Stable real-time performance

These characteristics align well with the requirements of a rocket launch control panel system.

6.4.5 Electrical Interface and Voltage Compatibility

The proposed hardware implementation operates using multiple voltage levels to ensure compatibility between different system components. The FPGA utilizes 3.3V logic levels, while several external modules like sensors, relay drivers, and display components operate using 5V supply levels. High-power devices such as actuators and relay-controlled systems are powered using an independent 12V external power supply.

Proper interfacing techniques are used to ensure safe communication between components operating at different voltage levels. Signal compatibility is achieved using interface circuits such as relay modules and level-protection components, preventing electrical damage to the FPGA and ensuring stable system operation.

6.5 Final Justification for FPGA Selection

Based on the comparison of available alternatives, the **Xilinx Artix-7 FPGA (Basys 3)** was selected as the primary implementation platform because it provides the best balance between performance, flexibility, and real-time responsiveness.

While the **STM32F407VG** represents the strongest MCU alternative, the **FPGA** offers parallel processing capability, precise hardware control, and superior real-time performance, which makes it the most suitable choice for implementing a reliable and responsive rocket launch control panel system.

7. Software Functional Block Diagram

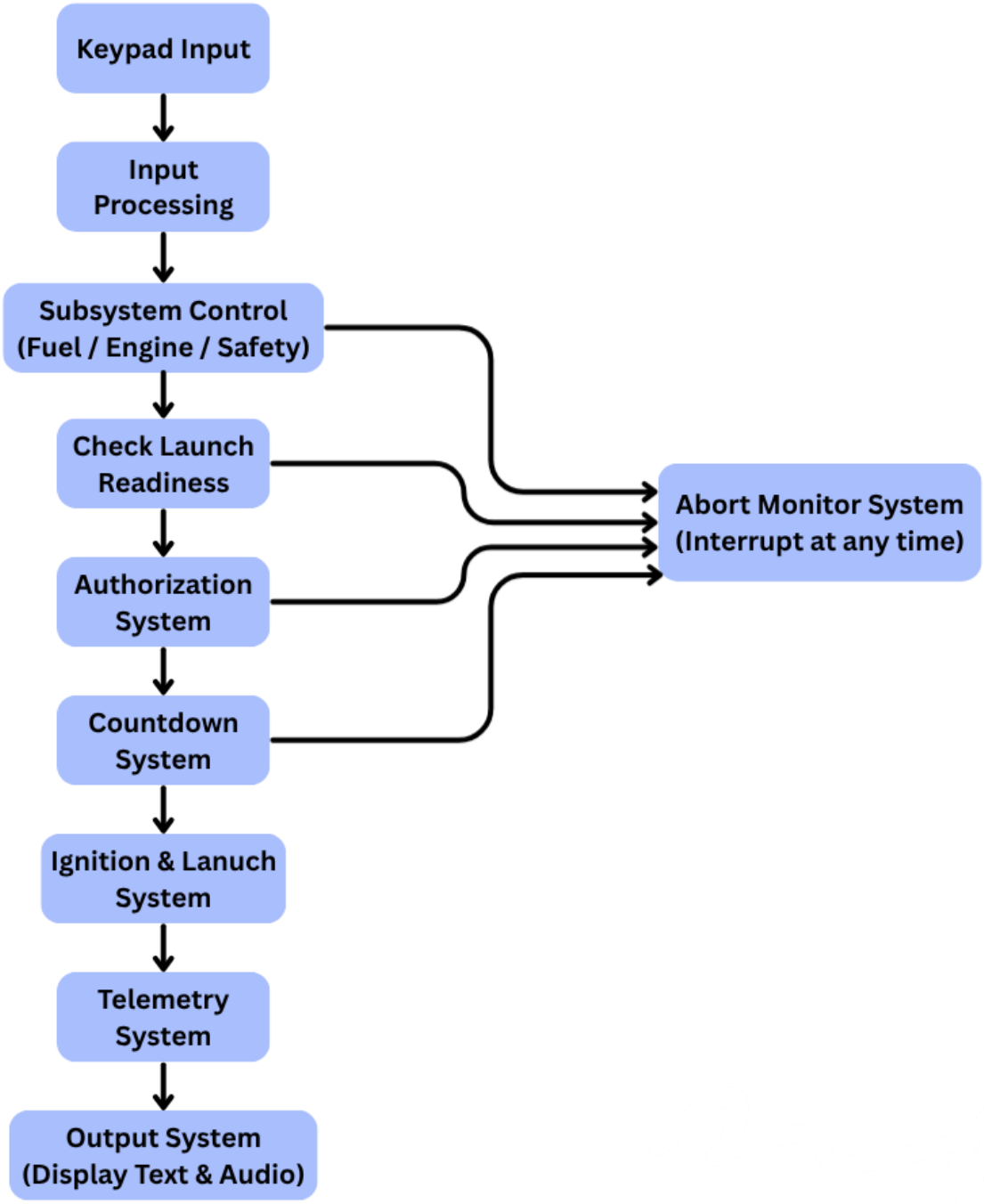


Figure 3: Software functional block diagram

8. Hardware Block Diagram

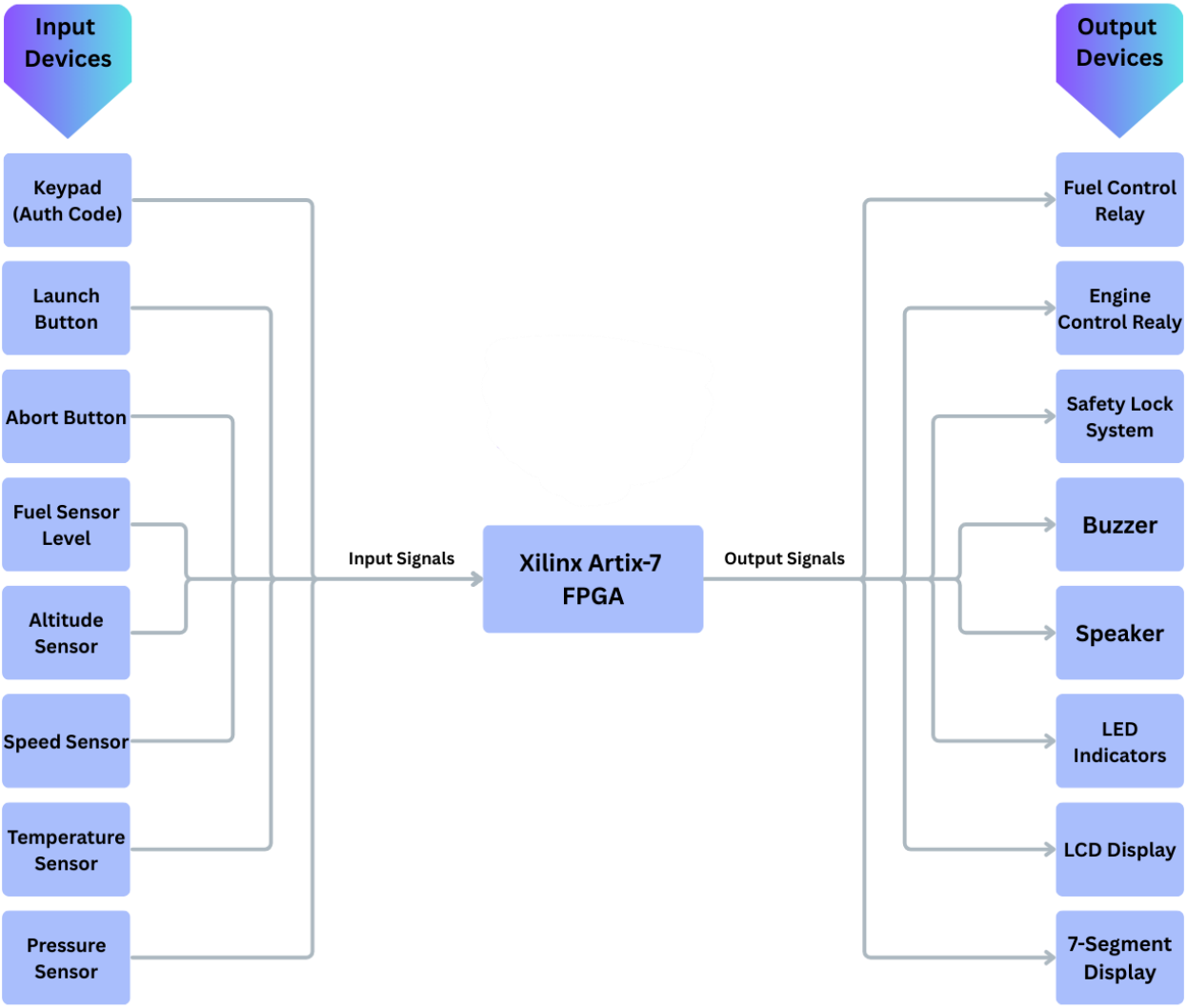


Figure 4: Hardware block diagram

The block diagram illustrates the interaction between the main system components. The FPGA acts as the central control unit and directly interfaces with all input and output devices through its configurable I/O pins.

Input devices such as sensors and control buttons send signals to the FPGA, where they are processed by the internal logic. Output devices such as the LCD display, LED indicators, buzzer, and subsystem controllers receive control signals generated by the FPGA.

This architecture enables efficient monitoring and control of the launch sequence with high performance and real-time responsiveness.

9. System Workflow (Flowchart)

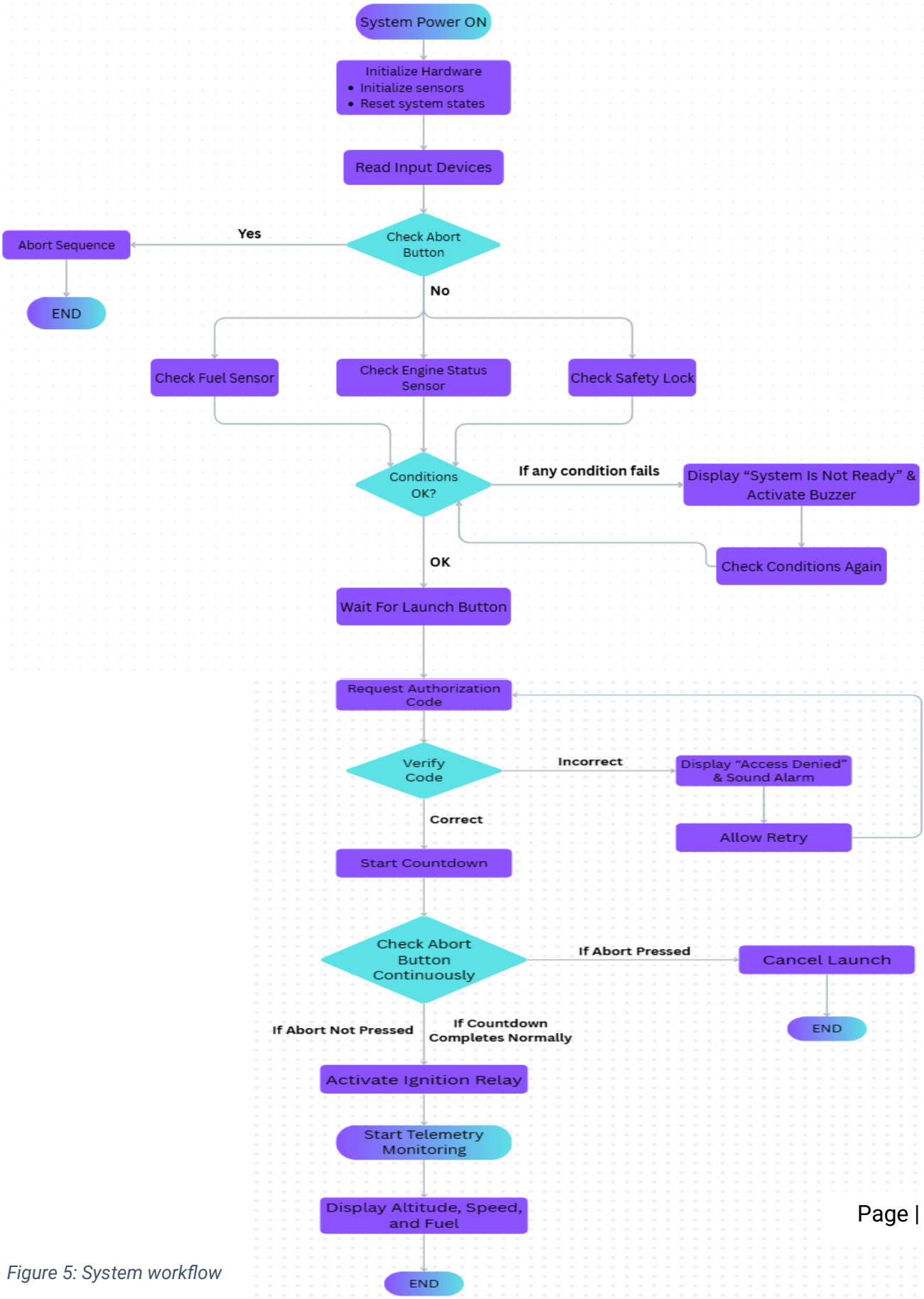


Figure 5: System workflow

10. Simulation Verification

To verify the correctness of the implemented system, several test cases were executed during the simulation. Each test case checks a specific scenario to ensure that the system behaves as expected under different conditions.

Test Case 1: Enable all systems + correct authorization code

Expected: The system should verify that all subsystems are enabled (Fuel, Engine, and Safety). After entering the correct authorization code, the system should display and say “Access Granted”, start the countdown sequence from 10 to 1, initiate ignition, and then display rocket telemetry data such as altitude, speed, and fuel level.

Actual:

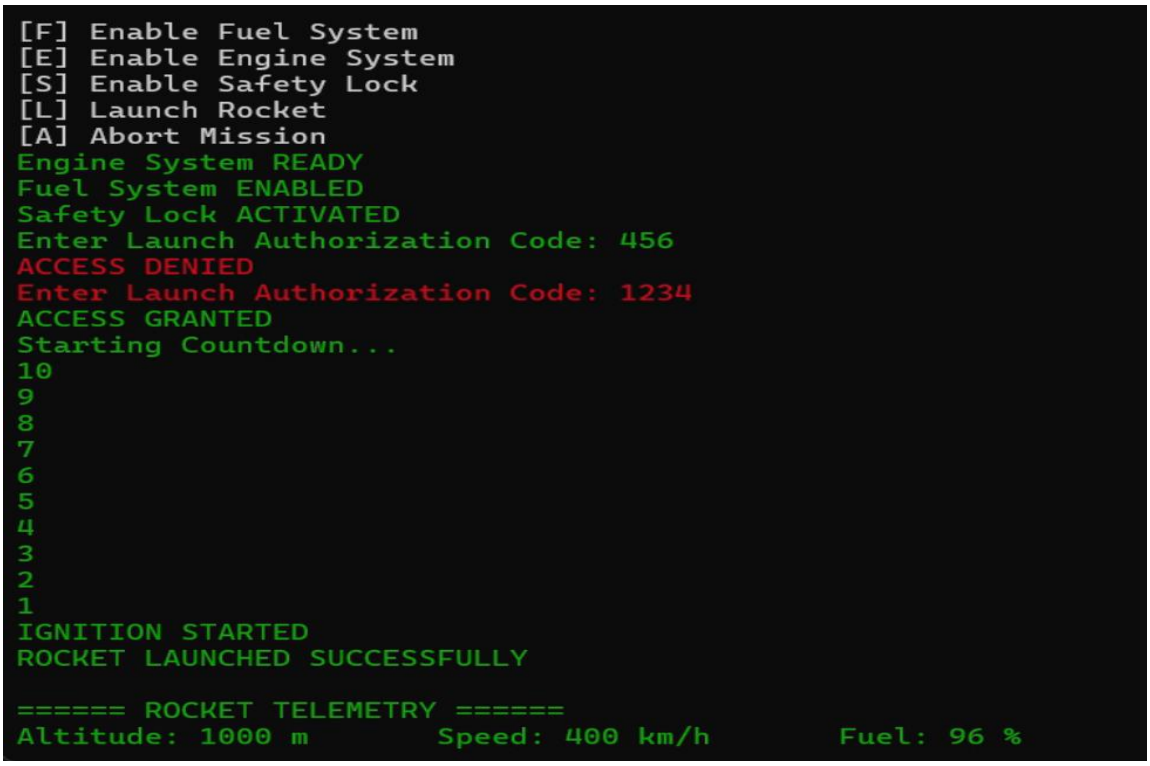
```
================================================
        ROCKET LAUNCH CONTROL PANEL
================================================

[F] Enable Fuel System
[E] Enable Engine System
[S] Enable Safety Lock
[L] Launch Rocket
[A] Abort Mission
Fuel System ENABLED
Safety Lock ACTIVATED
Engine System READY
Enter Launch Authorization Code: 1234
ACCESS GRANTED
Starting Countdown...
10
9
8
7
6
5
4
3
2
1
IGNITION STARTED
ROCKET LAUNCHED SUCCESSFULLY

====== ROCKET TELEMETRY ======
Altitude: 3500 m        Speed: 1400 km/h        Fuel: 86 %
```

Result: ✓ Passed

Test Case 2: Enable all systems + incorrect authorization code

Expected: The incorrect authorization code should be rejected by the system, then display and say “Access Denied”, activate the alarm, and prompt the user to enter the authorization code again.

Actual:

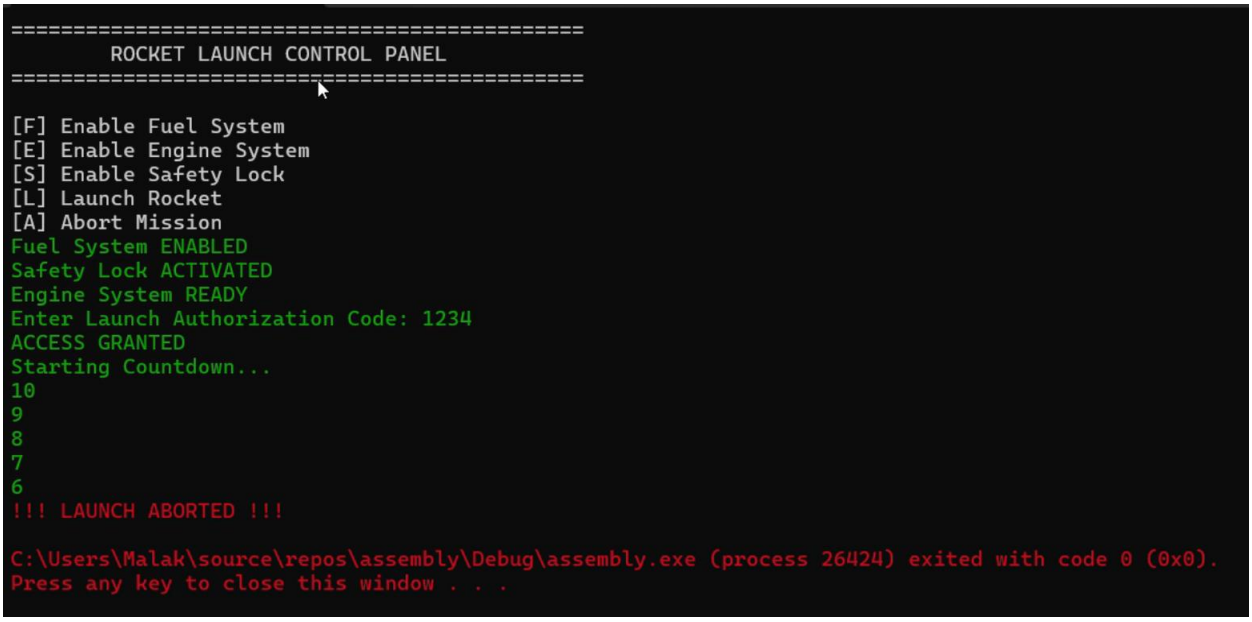


Result: ✓ Passed

Test Case 3: Abort during countdown

Expected: If the **Abort button** is pressed during the countdown, the system should immediately stop the countdown, cancel the launch sequence, display and say an **abort message** “!!! LAUNCH ABORTED !!!”, and activate the **alarm**.

Actual:

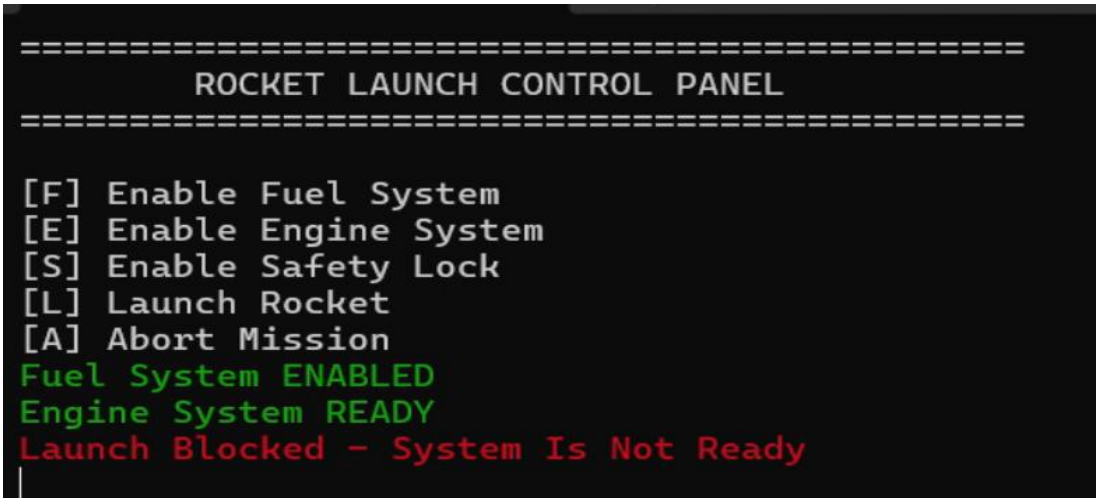


Result: ✓ Passed

Test Case 4: Enable Fuel + Engine but not Safety

Expected: The system should detect that the Safety Lock is not activated, as a result the system blocks the launch attempt, then display and say “Launch Blocked – System Is Not Ready”, and activate the alarm.

Actual:

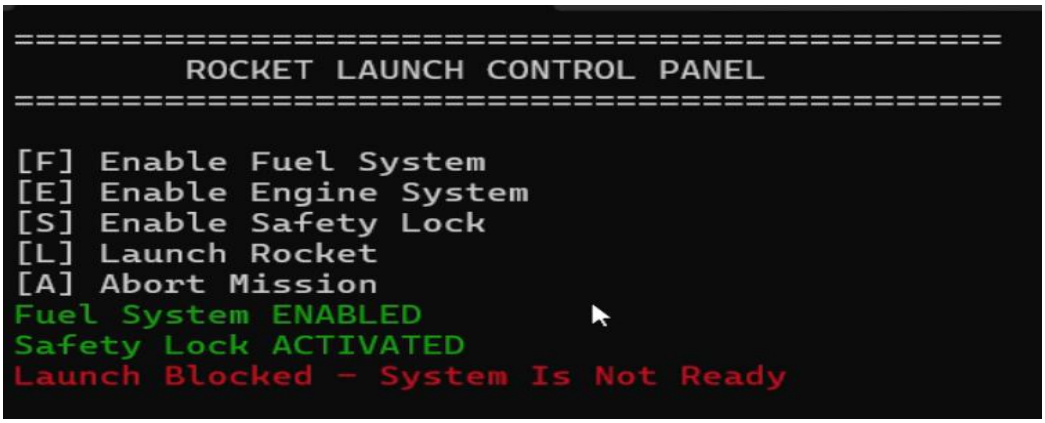


Result: ✓ Passed

Test Case 5: Enable Fuel + Safety but not Engine

Expected: The system should detect that the Engine System is not enabled, as a result the system blocks the launch attempt, then display and say “Launch Blocked – System Is Not Ready”, and activate the alarm.

Actual:

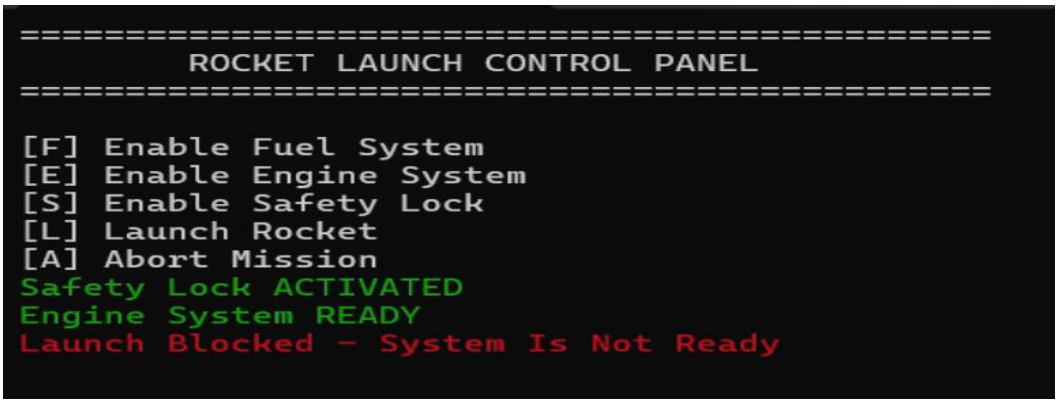


Result: ✓ Passed

Test Case 6: Enable Engine + Safety but not Fuel

Expected: The system should detect that the Fuel System is not enabled, as a result the system blocks the launch attempt, then display and say “Launch Blocked – System Is Not Ready”, and activate the alarm.

Actual:



Result: ✓ Passed

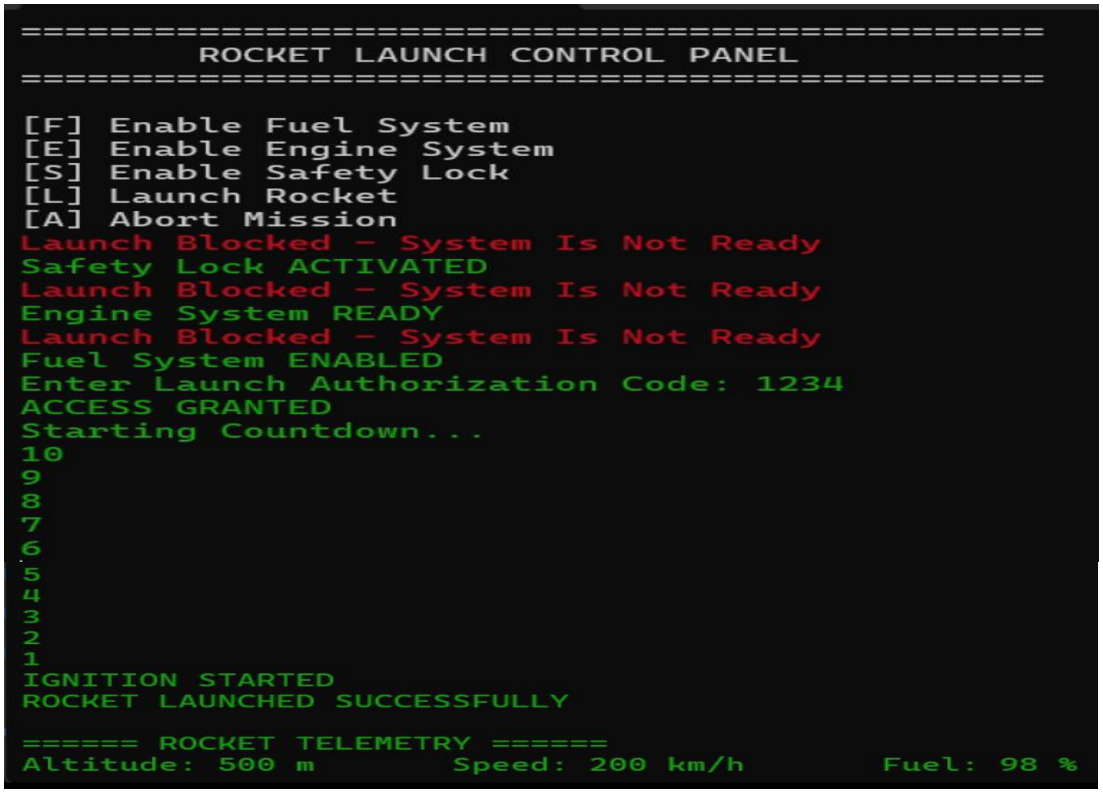
Test Case 7:

Steps for this test case:

1. Press Launch button without enabling any system
2. Enable Engine
3. Press Launch button
4. Enable Fuel
5. Press Launch button
6. Enable Safety
7. Press Launch button

Expected: The system should block the launch attempt whenever one or more required subsystems are not enabled. Only after all subsystems (Fuel, Engine, and Safety) are activated should the system allow the launch procedure and request the authorization code.

Actual:



Result: ✓ Passed

11. Conclusion

11.1 Summary

The Rocket Launch Control Panel project showed how to design and build a control panel using a platform based on an FPGA with assembly-level simulation. The Rocket Launch Control Panel system works like a real control panel by checking subsystems giving launch permission doing countdowns handling aborts and monitoring telemetry in time. It shows how a programmable digital system can do tasks while keeping safety, reliability and proper system checks in mind.

The main work was a software simulation using the Irvine32 library. The design was made to look like a real hardware setup. In this setup the FPGA connects to sensors, control inputs and output devices using I/O pins. Relay modules are used to control high-power subsystems. This design is flexible can respond in time and can be scaled up for complex embedded control systems.

11.2 Challenges

There were challenges when making this system. One big problem was making an assembly-language program that could handle many system states, such as turning on subsystems giving permission doing countdowns and taking abort actions. Using assembly language with the Irvine32 Library required an understanding of the available procedures and functions. Controlling the programs flow with instructions, loops and procedures needed careful planning to make sure the system worked correctly.

A big challenge was adding text-to-speech functionality for feedback from the Rocket Launch Control Panel system. Assembly language does not have built-in support for text-to-speech which made this requirement hard to do. At first two ways were considered. The first way was to build a text-to-speech engine from scratch in assembly language which would have needed hundreds of lines of complex code and increased development time. The second option was to use the Microsoft Speech API, which has built-in speech capabilities that can be accessed via external system calls. After thinking about both options the Microsoft Speech API approach was chosen as a practical and effective solution

allowing reliable speech output while keeping program complexity manageable.

Another challenge was turning the simulation into a hardware-oriented design. Choosing hardware components understanding voltage requirements and ensuring safe connections between the FPGA and I/O devices needed thorough research and careful system planning.

11.3 Outcomes

The project gave experience in assembly-language programming, digital system design and hardware-software integration. It greatly improved understanding of assembly-level programming concepts, including memory management, looping structures, state variables, conditional statements and modular programming with procedures. The process needed reference to technical documentation and programming resources to implement specific system behaviors correctly.

A significant outcome was gaining knowledge about selecting hardware and designing system architecture. The project involved evaluating hardware platforms, such as Microcontroller Units, Microprocessor Units, Systems-on-Chip and Field Programmable Gate Arrays. This research and evaluation process deepened the understanding of choosing the hardware based on system needs like processing power the number of I/O pins, real-time performance and scalability.

Considerable knowledge was also gained regarding FPGA technology, especially when selecting the Xilinx Artix-7 FPGA as the hardware platform. This included learning about FPGA vendors, device families and capabilities well as how FPGA systems offer changeable logic resources for real-time control applications.

Overall the project improved research skills, problem-solving abilities and system design thinking. It improved the capacity to solve engineering problems evaluate solutions and create efficient and reliable system designs based on both theoretical and practical factors.

12. Future Improvements

Possible future improvements for the system include:

- Integration with real sensors and actuators
- Real-time telemetry communication with a ground station
- Graphical monitoring interface
- Automatic fault detection for subsystem failures
- Secure multi-level authorization system
- Redundant safety control mechanisms
- Wireless sensor monitoring

Appendix A: Real Hardware Components

Device	Possible Hardware Model	Purpose
Microprocessor	• Xilinx Artix-7 FPGA • STM32F407 • Raspberry Pi Pico RP2040 (Dual-core ARM Cortex-M0+)	Main processing unit that executes the launch control program and manages system logic.
LCD Display	• JHD162A LCD • LM016L LCD • I2C LCD (PCF8574)	Displays system messages, status information, and rocket telemetry data.
7-Segment Display	• Common Anode 7-Segment (5161AS) • Common Cathode 7-Segment (3461BS) • Basys 3 Built-in 7-Segment Display	Displays the countdown sequence before rocket launch.
Keypad	• 4×4 Matrix Keypad • 3×4 Matrix Keypad	Allows the operator to enter the launch authorization code.
Buzzer	• Active Piezo Buzzer (5V) • Passive Piezo Buzzer	Generates alarm signals for system errors or mission abort events.
LED Indicators	• 5mm Standard LED Indicator • RGB LED	Provide visual indication of system states such as READY, ERROR, or ACTIVE systems.
Altitude Sensor	• BMP280 Pressure Sensor • BMP180 Pressure Sensor • BME280 Pressure Sensor	Measures altitude during rocket flight for telemetry monitoring.
Speed Sensor	• Hall Effect Sensor (A3144) • Optical Encoder Sensor • IR Speed Sensor	Measures rocket velocity or rotational speed for telemetry data.
Fuel Level Sensor	• Ultrasonic Level Sensor (HC-SR04) • Capacitive Fuel Level Sensor • Float Level Sensor	Detects and measures the fuel level in the rocket tank.
Control Buttons	• Panel Mount Push Buttons	Used for Launch, Abort activation commands.

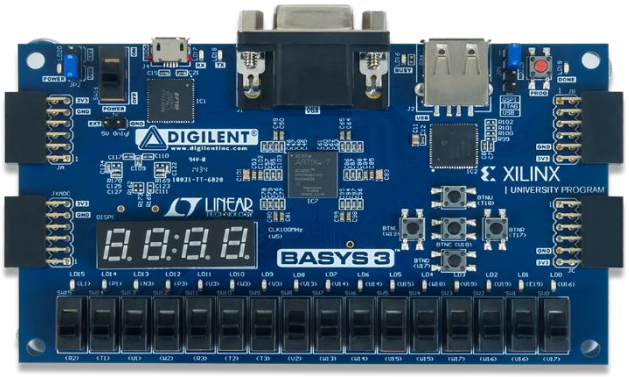
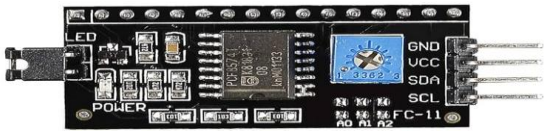
	• Tactile Push Button (6×6mm) • Omron B3F Push Button • Basys 3 Built-in Buttons	
Temperature Sensor	• LM35 Temperature Sensor • DS18B20 Digital Temperature Sensor • DHT11 Temperature Sensor • BME280 Sensor	Monitors engine temperature to ensure safe operation.
Pressure Sensor	• MPX4115 Pressure Sensor • BMP280 Pressure Sensor • BME280 Pressure Sensor	Monitors fuel line pressure and engine chamber pressure.
Fuel & Engine Control Relay	• 5V Relay Module (SRD-05VDC-SL-C) • Omron G3MB SSR • 1-Channel Relay Module	Activates or deactivates the rocket engine system.
Safety Lock System	• Electromagnetic Solenoid Lock (12V) • Servo Motor (SG90) • Electromagnetic Lock	Prevents launch unless all safety conditions are satisfied.
Speaker	• 8Ω 0.5W Mini Speaker • LM386 Audio Module • PAM8403 Amplifier + Speaker	Generates voice announcements and system notifications

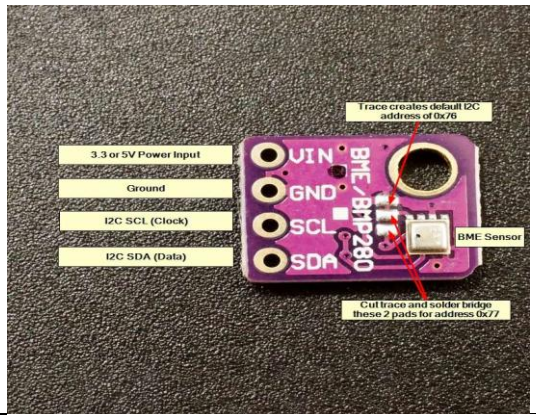
These hardware components represent possible real devices that could be used if the system were implemented as a physical embedded system rather than a simulation. And they were selected because they can interface with Xilinx Artix-7 Basys 3 board to communicate with sensors, control relays, and display devices through digital input/output signals.

Appendix B: Assembly Code

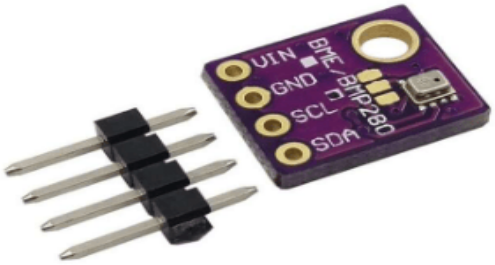
Function / Instruction	Purpose
ReadKey	It reads a key input from the keyboard
ReadDec	It reads a decimal input from the keyboard
Gotoxty	It moves the cursor to a specific row and specific column
SetTextColor	It changes the color of the text in the console
Delay	It delays the execution for “eax” milliseconds (ms)
Beep	It plays a beep sound within a given frequency and duration (for errors and mission abort)
ShowTelemetry	This displays the rocket info. after launching “altitude, speed, and fuel” on the screen in real time, which updates every one second until fuel is less than 10%
CheckAbort & AbortNow	It checks if the user pressed “a” to abort; if yes, it immediately aborts the mission, then it displays an abort message + beep sound then exits the program
Ignition	It handles the ignition sequence after countdown, it displays a success message + speech and then it shows the telemetry info. of the rocket
Alarm	It plays 3 consecutive beeps for errors or alerts or mission abort
Launch	It checks if the system is ready (Fuel, Engine, and Safety are ON) then it asks for the authorization code; if correct, countdown starts with speech then ignition starts
Messages	They are strings that are displayed for control panel, system states, launch, abort, telemetry
States	This checks the “fuelState”, “engineState”, and “safetyState” it they are on or off (“f”: fuelState on, “e”: engineState on, “s”: safetyState on)
Auth Code	This requires entering an auth code after ensuring that all the states are on to start countdown
Speech commands	Each message appears on the console has a corresponding PowerShell speech command

Appendix C: Hardware Models

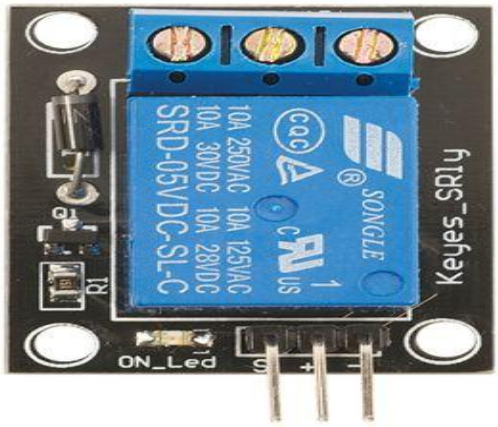
Hardware Device	Model Name
	**FPGA Xilinx Artix-7 Basys 3 Board**
	**5mm LED Indicators**
 	**20 × 4 I2C LCD Display**
	**Active Piezo Buzzer**



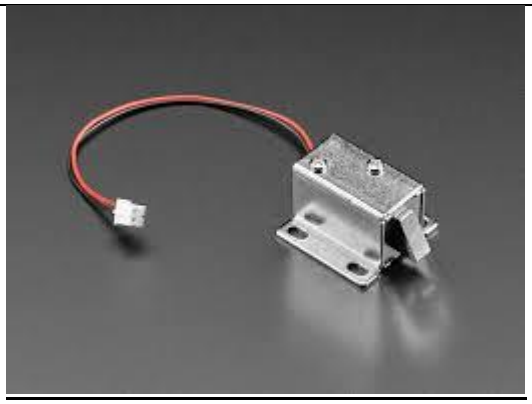
BME 280 I2C Humidity – Temperature – Pressure Sensor



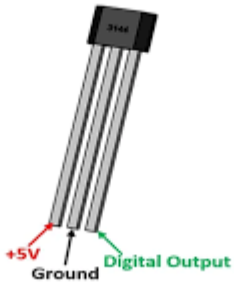
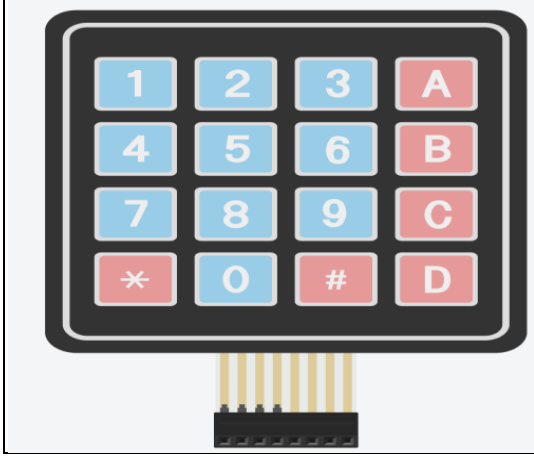
BMP 280 Barometric Pressure & Altitude Sensor

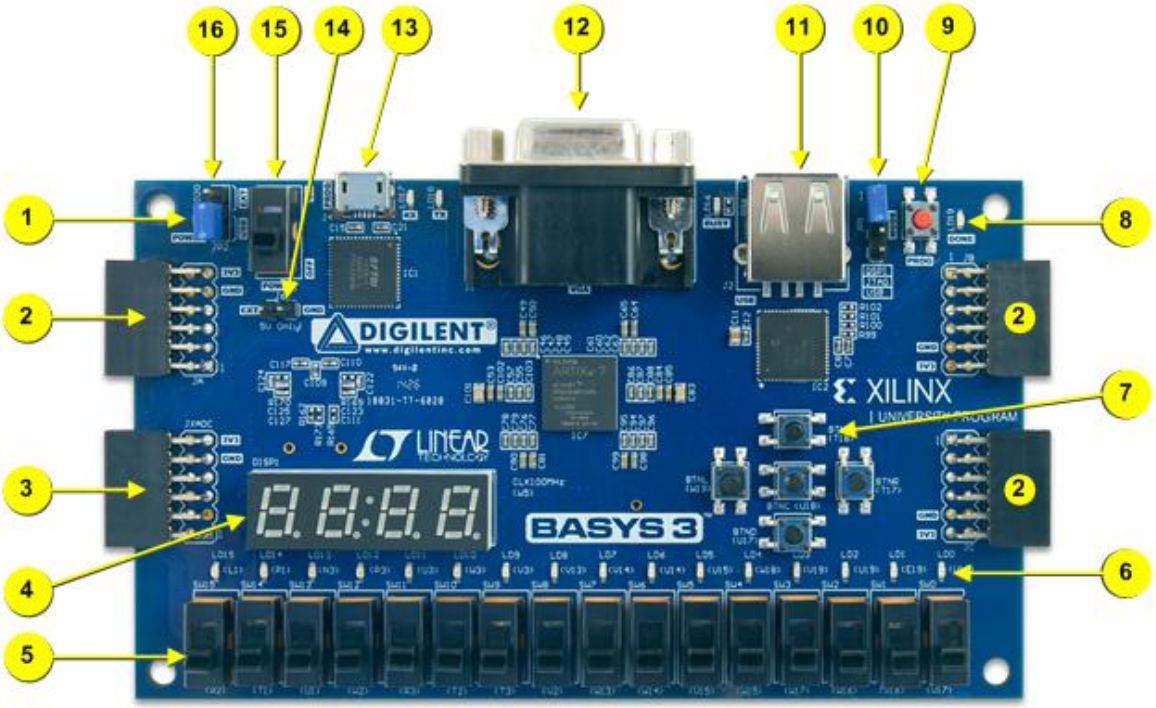


Relay Board (Relay + Transistor + Diode + Resistor)



Solenoid Lock

	**HC-SR04 Ultrasonic Fuel Level Sensor**
	**A3144 Hall Effect Speed Sensor**
	**LM386 Speaker**
	**4 × 4 Matrix Keypad**



Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod connector(s)	10	Programming mode jumper
3	Analog signal Pmod connector (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/ JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

References

[1] K. R. IRVINE, Assembly Language for x86 Processors (6th edition), Prentice Hall, 2010.

[2] D. Inc., "Basys 3™ FPGA Board Reference Manual," Digilent, Inc., Pullman, WA, April 8, 2016 .

[3] X. Inc., "7 Series FPGAs Data Sheet: Overview," Xilinx Inc., September 8, 2020.

[4] X. Inc., "XA Artix-7 FPGAs Data Sheet: Overview," Xilinx Inc., November 15, 2017.

[5] AMD, "Artix 7 FPGAs Data Sheet: DC and AC Switching Characteristics," AMD, July 3, 2024.

[6] R. P. Ltd, "RP2040 Datasheet," Raspberry Pi Ltd, February 20, 2025 .

[7] Y. Shi and H. Liu, " Beginner's Guide for Raspberry Pi Pico".

[8] STMicroelectronics, RM0090 Reference Manual, STMicroelectronics, June 2024.